Time Series Modeling - Bihag

Raphael Bihag

2026-03-31

Time Series Modeling

Introduction

This R notebook explores the R built-in dataset Lake Huron, which consists of the annual measurements of the level, in feet, of Lake Huron 1875–1972. This demonstrates the five main parts of time series analysis, from data wrangling to modeling and evaluation.

Data Wrangling

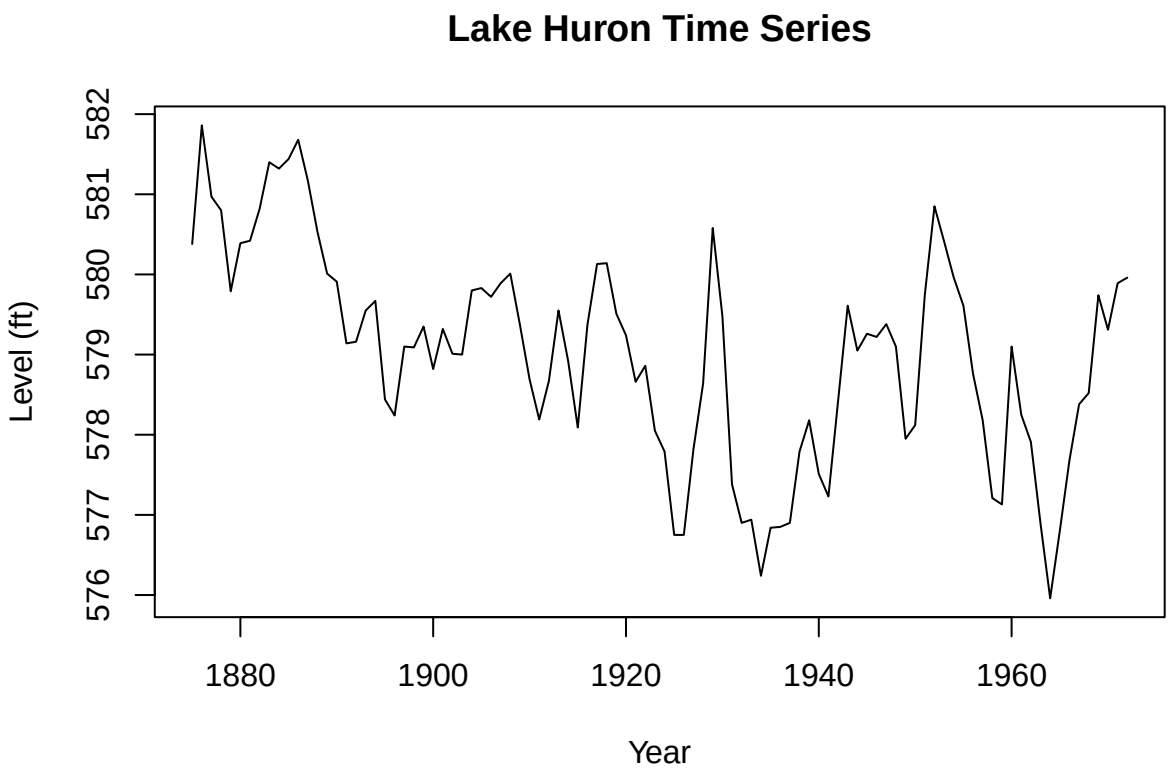
Load Required Libraries

```
library(forecast)
library(tseries)
library(ggplot2)
library(urca)
library(TSA)
library(dplyr)
```

Assess the Data

The dataset used in this analysis is the Lake Huron dataset, which is available natively in R. Let's plot it and get its summary.

```
# Loads dataset to a variable
lakehuron <- LakeHuron

# plots the dataset
plot(lakehuron, main="Lake Huron Time Series", xlab="Year", ylab="Level (ft)")
```




```
# Prints out a few lines of the dataset
lakehuron
```

```
## Time Series:
## Start = 1875
## End = 1972
## Frequency = 1
##  [1] 580.38 581.86 580.97 580.80 579.79 580.39 580.42 580.82 581.40 581.32
## [11] 581.44 581.68 581.17 580.53 580.01 579.91 579.14 579.16 579.55 579.67
## [21] 578.44 578.24 579.10 579.09 579.35 578.82 579.32 579.01 579.00 579.80
## [31] 579.83 579.72 579.89 580.01 579.37 578.69 578.19 578.67 579.55 578.92
## [41] 578.09 579.37 580.13 580.14 579.51 579.24 578.66 578.86 578.05 577.79
## [51] 576.75 576.75 577.82 578.64 580.58 579.48 577.38 576.90 576.94 576.24
## [61] 576.84 576.85 576.90 577.79 578.18 577.51 577.23 578.42 579.61 579.05
## [71] 579.26 579.22 579.38 579.10 577.95 578.12 579.75 580.85 580.41 579.96
## [81] 579.61 578.76 578.18 577.21 577.13 579.10 578.25 577.91 576.89 575.96
## [91] 576.80 577.68 578.38 578.52 579.74 579.31 579.89 579.96
```

The output shows that the data is yearly (frequency=1) and ranges from 1875 - 1972. This data is already cleaned and ready for analysis. Before doing our time series modeling, we have to test for the data's stationarity.

Stationarity Test & Transformation

We are going to test for the data's stationarity using the Augmented Dickey-Fuller test, which tests the null hypothesis that the data is non-stationary.

```
adf.test(lakehuron)
```

```
##
##  Augmented Dickey-Fuller Test
##
```

```
## data:  lakehuron
## Dickey-Fuller = -2.7796, Lag order = 4, p-value = 0.254
## alternative hypothesis: stationary
```

The result of the Augmented Dickey-Fuller test shows that the p-value is 0.254 which is greater than the 5% level of significance. Thus, we fail to reject the null hypothesis; the dataset is non-stationary.

Because the dataset is non-stationary, we apply transformations to make it stationary. For this case, we difference the dataset and check if it becomes stationary.

```
lakehuron_diff <- diff(lakehuron, lag=1)
adf.test(lakehuron_diff)
```

```
## Warning in adf.test(lakehuron_diff): p-value smaller than printed p-value
```

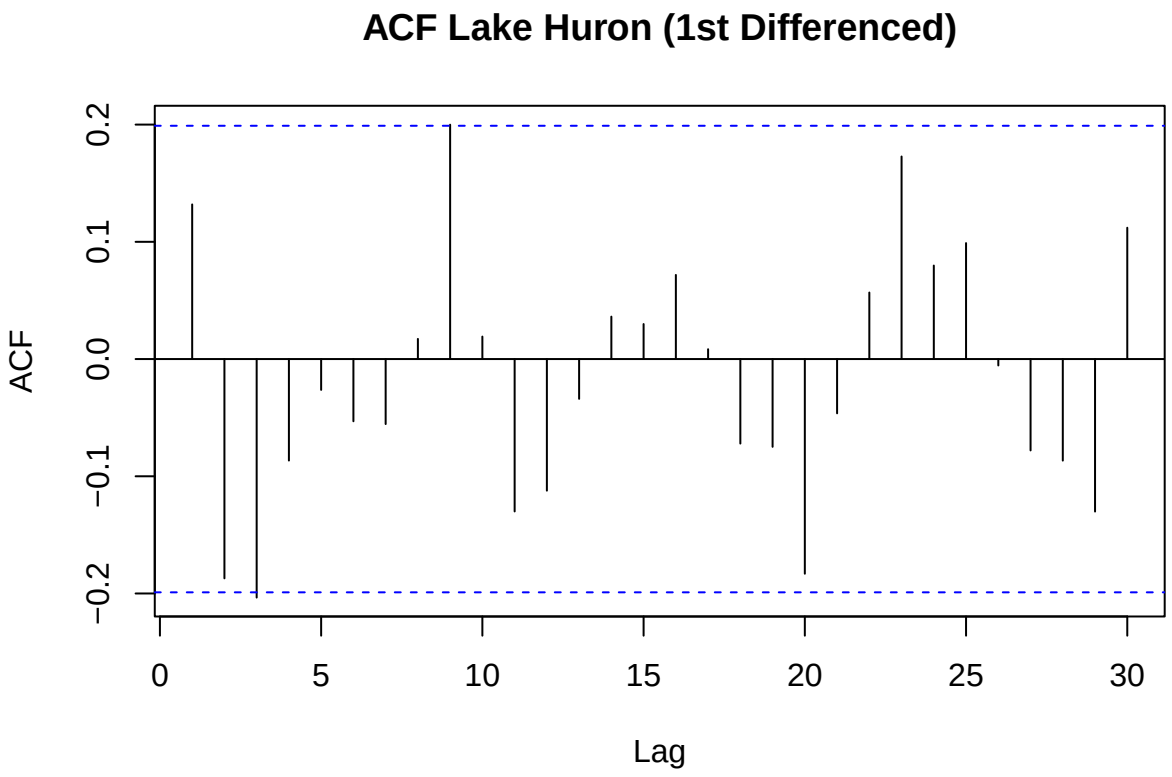
```
##
##  Augmented Dickey-Fuller Test
##
## data:  lakehuron_diff
## Dickey-Fuller = -5.4687, Lag order = 4, p-value = 0.01
## alternative hypothesis: stationary
```

The data has become stationary after the first iteration of differencing. Thus, the correct level of integration for this data is 1. We are now set for analysis.

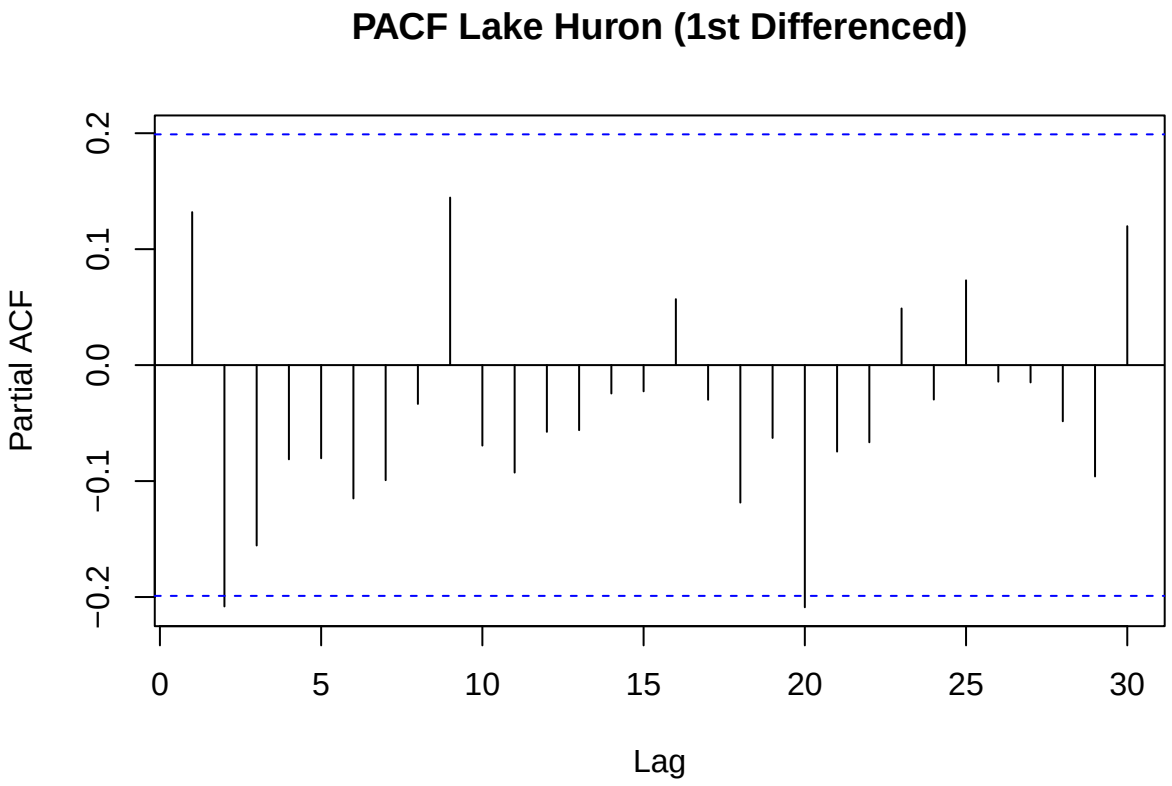
Model Identification

In this section, we perform ACF/PACF to identify the significant lag order for AR and MA components of the models. This step is important because it helps determine the appropriate values of p and q when modeling $AR(p)$ and $MA(q)$ processes.

```
acf(lakehuron_diff, main="ACF Lake Huron (1st Differenced)", lag.max = 30)
```



```r
pacf(lakehuron_diff, main="PACF Lake Huron (1st Differenced)", lag.max = 30)
```




```r
# computes the confidence bound for the dataset
conf_bound <- 1.96 / sqrt(length(lakehuron_diff))

# stores the acf plot data
acf_vals <- acf(lakehuron_diff, plot = FALSE, lag.max = 30)
pacf_vals <- pacf(lakehuron_diff, plot = FALSE, lag.max = 30)

# identifies which lags are significant from ACF and PACF
sig_acf <- which(abs(acf_vals$acf[-1]) > conf_bound)
sig_pacf <- which(abs(pacf_vals$acf) > conf_bound)

cat("Significant lags ACF: ", sig_acf, "\n")
```

```
## Significant lags ACF:  2 8
```

```r
cat("Significant lags PACF: ", sig_pacf, "\n")
```

```
## Significant lags PACF:  2 20
```

The results show that the significant lags from ACF (q) are 2 and 8, and that from PACF (p) are 2 and 20. With these values, we define our significant lag order for AR and MA components as variables to be used for later `ar_order <- 2` and `ma_order <- 2`. We will also define the order of integration (d) as 1 since we differenced the data once to make it stationary.

```r
ar_order <- 2
ma_order <- 2
d_value <- 1
```

Model Fitting

Before fitting our models, we have to split the data into train set and test set at 80-20 ratio.

Data Splitting

```
n <- length(lakehuron)  # holds the length of the dataset
train_size <- floor(0.8 * n)  # defines how many obs for the train size

# Data splitting
lh_train <- window(lakehuron, end = time(lakehuron)[train_size])
lh_test <- window(lakehuron, start = time(lakehuron)[train_size + 1])

# Output
cat("Lake Huron Split \n")
```

```
## Lake Huron Split
```

```
cat("> Training Set:", length(lh_train), "obs (", start(lh_train)[1], "-", end(lh_train)[1], ")\n")
```

```
## > Training Set: 78 obs ( 1875 - 1952 )
```

```
cat("> Test Set:", length(lh_test), "obs (", start(lh_test)[1], "-", end(lh_test)[1], ")\n")
```

```
## > Test Set: 20 obs ( 1953 - 1972 )
```

Model Fitting

Next, we autofit an ARIMA model using the AR and MA orders identified. The model is defined as $ARIMA(p, d, q)$ where p is the number of significant lags of PACF, q is the number of significant lags of ACF, and d is the order of integration.

```
# Variable to hold all of the results
results <- list()

# fitting the AR model
results$ar <- Arima(lh_train, order = c(ar_order, d_value, 0))

# fitting the MA model
results$ma <- Arima(lh_train, order = c(0, d_value, ma_order))

# fitting the ARMA model
results$arma <- Arima(lh_train, order = c(ar_order, d_value, ma_order))

# fitting the auto-ARIMA model
results$auto <- auto.arima(lh_train)
```

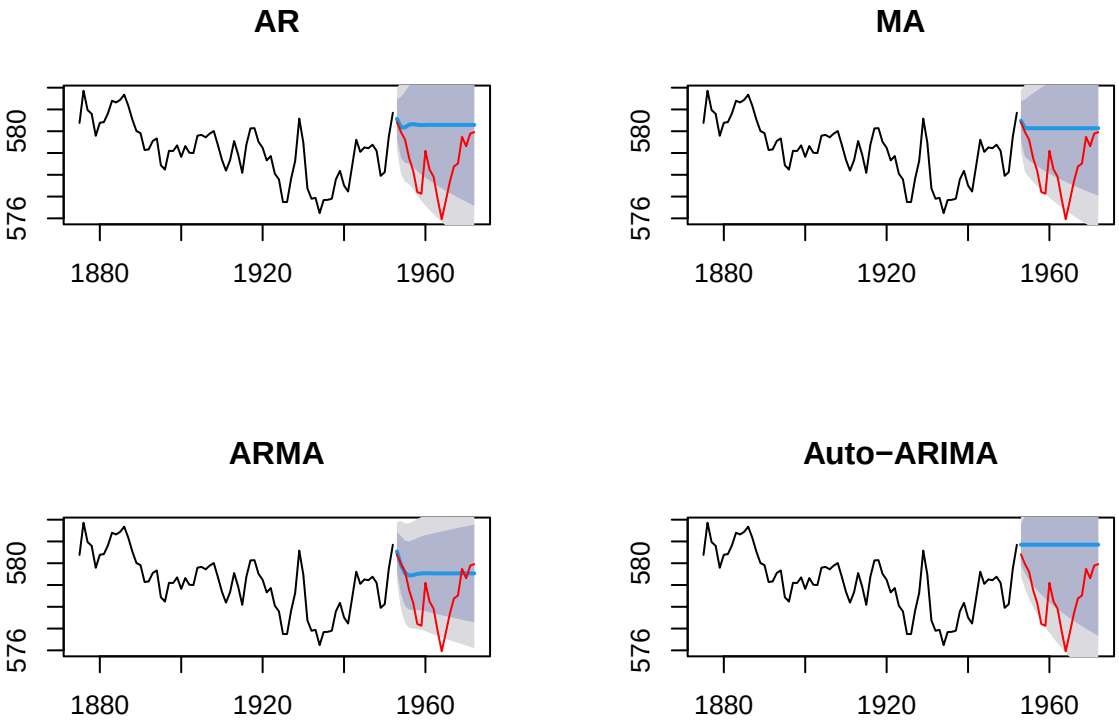
Forecasting

We have successfully fitted all our models. Next, let's forecast using these models, which we will use later to evaluate each model's accuracy. The forecast duration must be the same length as the test data.

```
forecasts <- list()

# Forecasting using AR
forecasts$ar <- forecast(results$ar, h = length(lh_test))
```

```r
# Forecasting using MA
forecasts$ma <- forecast(results$ma, h = length(lh_test))

# Forecasting using ARMA
forecasts$arma <- forecast(results$arma, h = length(lh_test))

# Forecasting using auto-ARIMA
forecasts$auto <- forecast(results$auto, h = length(lh_test))
```

```r
par(mfrow = c(2,2))
plot(forecasts$ar, main= "AR", ylim=range(lakehuron))
lines(lh_test,col= "red")

plot(forecasts$ma, main= "MA", ylim=range(lakehuron))
lines(lh_test,col= "red")

plot(forecasts$arma, main= "ARMA", ylim=range(lakehuron))
lines(lh_test,col= "red")

plot(forecasts$auto, main= "Auto-ARIMA", ylim=range(lakehuron))
lines(lh_test,col= "red")
```



These are the performances of the time series models. Plotted in black is the historical (train) data; in red is the test part of the historical data; and in blue is the forecast. For further analyis, we look at the Akaike Information Criterion (AIC) of each model to assess the best fit.

AIC Values

Akaike Information Criterion (AIC) is a metric for measuring the quality of a statistical model for a given data set, with lower AIC scores indicating better quality models. AIC is a relative estimator, and is useful in comparison with other AIC scores of the same data set.

```r
# To store all the AIC values
aic_values <- c(
  ar   = results$ar$aic,
  ma   = results$ma$aic,
  arma = results$arma$aic,
  auto = results$auto$aic
)

# Get the model with the least AIC
min_val   <- min(aic_values)
best_name <- names(which.min(aic_values))

cat("Least AIC:", best_name, " - ", min_val)
```

```
## Least AIC: arma  -  167.0691
```

Among the time series models, the ARMA model is found to have the least AIC, with a value of 167.07. We then use the `summary` function to get a closer look at the ARMA model created from our dataset.

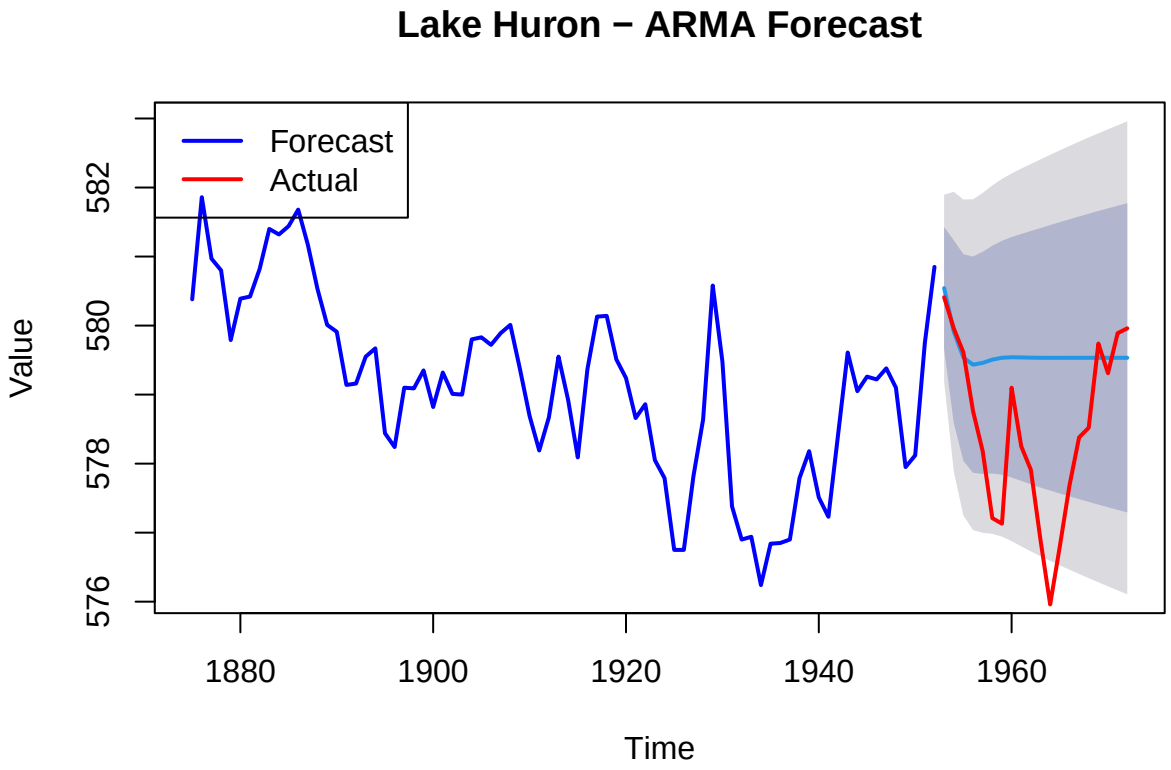
```r
summary(results$arma)
```

```
## Series: lh_train
## ARIMA(2,1,2)
##
## Coefficients:
##          ar1      ar2      ma1      ma2
##       0.7286  -0.2702  -0.6148  -0.1366
## s.e.  0.2809   0.2572   0.2850   0.2727
##
## sigma^2 = 0.4764:  log likelihood = -78.53
## AIC=167.07   AICc=167.91   BIC=178.79
##
## Training set error measures:
##                       ME      RMSE       MAE          MPE      MAPE      MASE
## Training set -0.01594172 0.6677292 0.5370447 -0.002850356 0.0927297 0.9592309
##                      ACF1
## Training set 0.002528463
```

Forecast Visualization and Diagnostic Checking

The best fit model is ARMA. Below is the forecast visualization of it.

```r
plot(forecasts$arma, main= "Lake Huron - ARMA Forecast",
     xlab = "Time",
     ylab = "Value",
     col = "blue", lwd="2")
lines(lh_test,col= "red", lwd = 2)
legend("topleft", legend = c("Forecast", "Actual"),
col = c("blue", "red"), lty = c(1, 1), lwd = c(2, 2))
```



For diagnostics, we are going to perform the Ljung-Box test to check whether the residuals are independently distributed (i.e., no autocorrelation). This test is required to be performed to assess if the model has correctly fitted the data with the right amount of lags. This is to test the null hypothesis H_0 that the residuals are independently/normally distributed against the alternative H_a that the residuals are not independently distributed (the model has failed to capture some underlying pattern).

```
model_residuals <- residuals(results$arma)
lb_test <- Box.test(model_residuals)
print(lb_test)
```

```
##
##  Box-Pierce test
##
## data:  model_residuals
## X-squared = 0.00049866, df = 1, p-value = 0.9822
```

The test shows a p-value=0.9822, which is greater than the level of significant of 5%. Thus, we have insufficient evidence to reject the null hypothesis—the residuals are normally distributed.

Summary of Results and Discussion

The plot of the Lake Huron data shows a visible trend but no strong seasonality. The trend component is found to be profound which resulted to the data being non-stationary. This non-stationary behavior of the data is confirmed by the ADF test, which resulted to p-value=0.254, leading us to fail to reject the null hypothesis. Because of this, first-order differencing was found to be sufficient to make the data stationary.

From the ACF and PACF plots, the significant lag order for each model was identified. The ACF plot found the lag order $q = 2$, and the PACF plot found the lag order $p = 2$. Although there are two significant lags from each of the plots, the smaller lag orders are chosen, as higher lags may be due to noise and could lead to overfitting.

Upon fitting the four time series models to the lag orders, the best fitted model based on the least AIC is

found to be the ARMA model `ARIMA(2,1,2)`.

The `summary` function shows that the equation of the ARMA model is given by,

$$\Delta\hat{y}_t = 0.7286\Delta y_{t-1} - 0.2702\Delta y_{t-2} + \epsilon_t - 0.6148\epsilon_{t-1} - 0.1366\epsilon_{t-2}$$

The coefficients of this model can be interpreted as follows:

1. The change in level from a year ago increases the current year's change in level by 72.86%.
2. The change in level from two years ago decreases the current year's change in level by 27.02%.
3. The change in shocks from last year decreases the current year's change in shocks by 61.48%.
4. The change in shocks from two years ago decreases the current year's change in shocks by 13.66%.

The diagnostic test revealed the normality of the residuals, which suggests good model specification—there is no autocorrelation in the data up to the specified number of lags, and the series behaves like white noise.

From the forecast visualization, we can see that most of the actual data are within the forecast bounds of the best fit model. The model was able to **accurately capture the next 2-3 years** but then deviated significantly after that due to sudden changes in lake levels that the model cannot capture.